

Facultad de Ciencias UNAM
Estructuras de Datos
Nota2: Ejemplos de Tipos de dato abstracto

Profesora: Virginia Teodosio Procopio
Ayudantes: Leonardo Gallo Guerrero
José Manuel Madrigal Ramírez

26 de Agosto de 2026

Repaso de TDA

Es muy importante enfatizar que los tipos de dato abstracto no son implementaciones. Esto quiere decir que no dicen nada sobre cómo es que las operaciones cumplen con el objetivo esperado. Y mucho menos están escritos en algún lenguaje de programación, por el contrario, los TDA son formalizaciones de un requerimiento funcional. Dicho de otro modo, un TDA sólo expresa qué comportamiento se espera de ciertas operaciones sobre un dominio particular.

Dicho comportamiento queda determinado por sus axiomas. En ellos también están implícitas las precondiciones y postcondiciones del TDA.

Ejemplo 1: TDA Contador

Comencemos con un contador de números naturales. ¿Qué comportamiento podríamos esperar de esta entidad? En primer lugar nos gustaría que pudiese incrementar su valor interno cada vez que lo indiquemos. Del mismo modo, nos gustaría poder reiniciar el contador en un momento dado. Para inicializar este TDA requeriremos de una operación que haga las veces de constructor, y finalmente, nos gustaría poder acceder al valor interno del contador.

Dicho lo anterior, concluimos que necesitamos cuatro operaciones para modelar el comportamiento de un *Contador*. Para poder formalizar su comportamiento en el siguiente TDA, también necesitaremos de una variable *n* que represente el valor interno del *Contador* y de una variable *C* para referirnos al *Contador* cuando sea necesario dentro de los axiomas de las operaciones:

Listing 1: Especificación del TDA Contador.

TDA Contador

REQUIERE

Natural

VARIABLES

n: Natural
C: Contador

OPERACIONES

crear : Natural -> Contador
incrementar : Contador -> Contador
reiniciar : Contador -> Contador
getValor : Contador -> Natural

AXIOMAS

```
[C1] getValor(crear(n)) = n
[C2] getValor(incrementar(C)) = getValor(C) + 1
[C3] getValor(reiniciar (C)) = 0
```

FIN Contador

Ejemplo 2: TDA Natural

Observa que en nuestro TDA Contador requeríamos de un tipo de dato que aún no hemos definido: el TDA Natural. No dejemos pasar más tiempo y definámoslo. Para empezar necesitamos de una especificación formal que por suerte Giuseppe Peano ya proporcionó mediante una axiomatización para los números naturales en el siglo XIX. Seguramente ya has escuchado hablar de ella, en caso de que no, la enunciaremos a continuación:

1. Existencia del cero:

$$0 \in \mathbb{N}$$

El cero es un número natural.

2. Inyectividad del sucesor:

$$\forall n, m \in \mathbb{N}, \quad (S(n) = S(m) \implies n = m)$$

Si dos números naturales tienen el mismo sucesor, entonces son iguales.

3. El cero no es sucesor de nadie:

$$\forall n \in \mathbb{N}, \quad S(n) \neq 0$$

No existe ningún número natural cuyo sucesor sea 0.

4. Cerradura de la operación sucesor:

$$\forall n \in \mathbb{N}, \quad S(n) \in \mathbb{N}$$

Si n es un número natural, su sucesor $S(n)$ también es un número natural.

5. Axioma de inducción matemática:

$$\forall P \subseteq \mathbb{N}, \quad ((0 \in P \wedge (\forall n \in \mathbb{N}, n \in P \implies S(n) \in P)) \implies P = \mathbb{N})$$

Si un conjunto de naturales P contiene al 0, y siempre que contiene a un elemento n contiene también a su sucesor $S(n)$, entonces P contiene a todos los números naturales.

Como bien habrás observado esta axiomatización construye el conjunto de los naturales a partir de una operación y una constante. Bajo esta idea un número natural n queda definido aplicando la operación $\text{suc}()$ n veces al 0:

$$\begin{aligned} 1 &= \text{suc}(0) \\ 2 &= \text{suc}(\text{suc}(0)) \\ 3 &= \text{suc}(\text{suc}(\text{suc}(0))) \\ &\vdots \\ n &= \underbrace{\text{suc}(\text{suc}(\dots(\text{suc}(0))\dots))}_{n \text{ veces}} \end{aligned}$$

Nos centraremos en reflejar los cuatro primeros axiomas en nuestro TDA `Natural`. Por otro lado, además de la operación `suc()`, necesitamos una noción de igualdad dentro de nuestro sistema pues de otro modo sería imposible expresar propiedades como la inyectividad de la función `suc()` o la idea de que el 0 no es sucesor de nadie. De modo que nuestro TDA `Natural` también contemplará una operación `igual()` que recibe dos parámetros y devolverá un `Booleano`.

Sin más dilación, veamos cómo queda formalizado el comportamiento de los números naturales en el siguiente TDA:

Listing 2: Especificación del TDA Natural.

```
TDA Natural

REQUIERE
    Booleano

CONSTANTES
    0: Natural

VARIABLES
    x, y: Natural

OPERACIONES
    suc : Natural -> Natural
    igual : Natural × Natural -> Booleano

AXIOMAS
    [N1] igual(0,0) = VERDADERO
    [N2] igual(x, y) = igual (y, x)
    [N3] igual(suc(x), 0) = FALSO
    [N4] igual(suc(x), suc(y)) = igual(x,y)

FIN Natural
```

Observa cómo los axiomas de Peano permean de formas distintas en nuestro TDA `Natural`. El primer axioma, sobre la existencia del 0, queda fijo en la definición de las constantes, donde estipulamos que el 0 es de tipo `Natural`.

El hecho de que todo natural tiene un sucesor queda implícito en la definición de la operación mientras que su inyectividad queda reflejada en el axioma *N4*. Por otro lado, el axioma *N3* encierra el hecho de que 0 no es sucesor de nadie. Finalmente hay que notar que nuestro TDA no incluye el axioma de inducción pues su formalización incluye detalles técnicos que rebasan los límites del curso.

Ahora bien, los axiomas de este tipo de dato puede extenderse para cubrir la noción de suma, por ejemplo:

Listing 3: Axiomas de suma para el TDA Natural.

```
OPERACIONES
    suma: Natural × Natural -> Booleano

AXIOMAS
    [N5] suma (x, 0) = x
    [N6] suma (x, suc(y)) = suc (suma (x, y))
    [N7] suma (x, y) = suma (y, x)
```

Ejemplo 3: TDA Usuario

Hemos visto algunos ejemplos de TDA un tanto teóricos y ejemplos prácticos que podríamos decir «de juguete» como el TDA *Persona* y el TDA *Contador*. Pues bien, ahora abordaremos un TDA cuyo objetivo será modelar al usuario de una aplicación y nos servirá para nuestra clase de laboratorio.

El TDA *Usuario*, heredará características del TDA *Persona*, que dejamos aquí una vez más para rápida referencia:

```
TDA Persona

REQUIERE
    Cadena, Natural

OPERACIONES
    crear : Cadena  $\times$  Natural  $\rightarrow$  Persona
    getNombre : Persona  $\rightarrow$  Cadena
    getEdad : Persona  $\rightarrow$  Natural
    setNombre : Persona  $\times$  Cadena  $\rightarrow$  Persona

VARIABLES
    p : Persona
    n : Cadena
    e : Natural

AXIOMAS
    [P1] getNombre(crear(n, e)) = n
    [P2] getEdad(crear(n, e)) = e
    [P3] getNombre(setNombre(p, n)) = n
    [P4] getEdad(setNombre(p, n)) = getEdad(p)

FIN Persona
```

Listing 4: TDA Persona.

¡Perfecto! El TDA *Persona* nos permite almacenar el nombre y la edad. Ahora necesitamos pensar en qué características añadirle para que se trate del usuario de una aplicación. El usuario más básico de cualquier aplicación, que requiera crear una cuenta, está asociado a un correo electrónico y a una contraseña. Este par de atributos deben ser verificados por la aplicación para permitir el acceso del usuario. Además permitiremos que el usuario cambie su contraseña. Veamos cómo se formaliza el comportamiento de esta especificación en el TDA *Usuario*:

```
TDA Usuario extiende Persona

REQUIERE
    Booleano

OPERACIONES
    crearUsuario : Cadena  $\times$  Entero  $\times$  Cadena  $\times$  Cadena  $\rightarrow$  Usuario
    getCorreo : Usuario  $\rightarrow$  Cadena
    setCorreo : Usuario  $\times$  Cadena  $\rightarrow$  Usuario
    validar : Usuario  $\times$  Cadena  $\times$  Cadena  $\rightarrow$  Booleano
    setClave : Usuario  $\times$  Cadena  $\times$  Cadena  $\rightarrow$  Usuario
```

VARIABLES

u : Usuario
e : Natural
n, correo, correoNvo, clave, claveVieja, claveNva : Cadena

AXIOMAS

```
[U1] getCorreo(crearUsuario(n, e, correo, clave)) = correo
[U2] getNombre(crearUsuario(n, e, correo, clave)) = n
[U3] getEdad(crearUsuario(n, e, correo, clave)) = e

[U4] getCorreo ( setCorreo (u, correoNvo)) = correoNvo
[U5] getNombre ( setCorreo (u, correoNvo)) = getNombre(u)
[U6] getEdad ( setCorreo (u, correoNvo)) = getEdad(u)

[U7] validar(crearUsuario(n, e, correo, clave), correo, clave) = Verdadero
[U8] Si
    or(not(igual (correo, correoNvo))) = Verdadero
    Entonces:
        validar(crearUsuario(n, e, correo, clave), correoNvo, clave) = Falso
[U9] Si
    or(not(igual (clave, claveNva))) = Verdadero
    Entonces:
        validar(crearUsuario(n, e, correo, clave), correo, claveNva) = Falso

[U10] Si
    validar(u, getCorreo(u), claveVieja) = Verdadero
    Entonces
        getNombre(setClave(u, claveVieja, claveNva)) = getNombre(u)
        getEdad(setClave(u, claveVieja, claveNva)) = getEdad(u)
        getCorreo(setClave(u, claveVieja, claveNva)) = getCorreo(u)
        validar((setClave(u, claveVieja, claveNva)), getCorreo(u), claveNva) = Verdadero

[U11] Si
    validar(u, getCorreo(u), claveVieja) = Falso
    Entonces
        (setClave(u, claveVieja, claveNva) = u
```

Listing 5: TDA Usuario.

¡VAYA!
MUCHO TDA USUARIO
...
TOMEMOS UN RESPIRO



Ahora analicemos brevemente los axiomas. Del *U1* al *U6* estamos tratando con el comportamiento del acceso a los atributos del usuario y la posibilidad de modificarlos. Consideramos que no sería buena práctica dejar al descubierto la contraseña del usuario, digamos, mediante una operación `getClave` porque eso la pondría a disposición de cualquier otro programa.

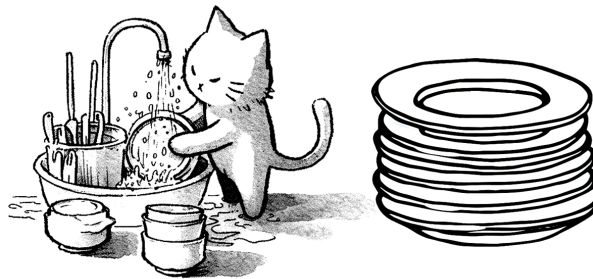
Los axiomas *U7* al *U9*, indican cómo debería comportarse la validación de la información del usuario. En particular el axioma *U7* nos dice que si el usuario ingresa correctamente su clave y su correo al sistema será validado. Por su parte los axiomas *U8* y *U9* plantean los casos en el que el usuario ingresa mal su correo o su clave respectivamente.

El axioma *U10* describe el comportamiento del cambio de contraseña cuando el usuario pudo validar sus credenciales adecuadamente. En realidad este axioma es un resumen de cuatro axiomas que dependen de una misma condición, a saber, que el usuario haya ingresado correctamente su correo y su contraseña antigua.

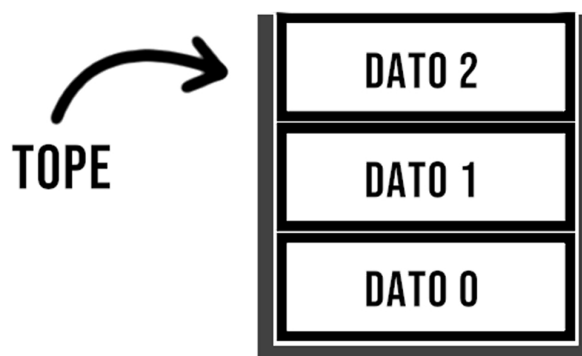
Finalmente el axioma *U11* describe lo que sucedería si el usuario no pudo validar sus credenciales en cuyo caso la operación de cambio de contraseña `setClave` no tendrá efecto sobre los datos del usuario.

Ejemplo 4: TDA Pila

Para finalizar estas notas introduciremos un último ejemplo referente a una conocida estructura de datos: La Pila. Más adelante en nuestro curso analizaremos a detalle esta estructura, por ahora basta decir que, metafóricamente, la pila organiza los datos del mismo modo en que nosotros colocamos los platos en la tarja antes de lavarlos, es decir, uno encima del otro. Además, el primer plato que lavamos suele ser el último que colocamos en la tarja, es decir, el que quedó por encima del resto.



Bajo esta intuición podríamos abstraer la idea de pila en el siguiente diagrama:



Ahora bien, las operaciones con las que cuenta nuestra *Pila*, y que detallaremos en la siguiente página son:

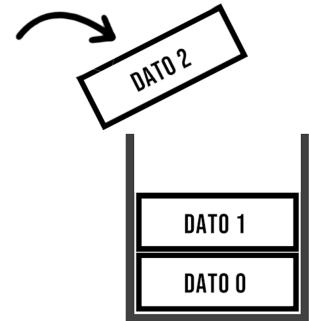
- Push
- Pop
- Top
- esVacía

Operaciones de una Pila

Para comenzar a detallar las operaciones de una pila y que podamos contruir nuestro TDA Pila, asumiremos que los datos que ingresaremos en nuestra *Pila* pertenecen a un conjunto A dado. Esto se denota como $Pila[A]$.

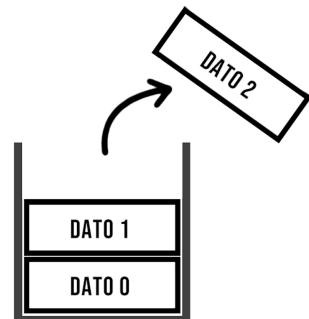
$Push : Pila[A] \times A \rightarrow Pila[A]$

Esto es, dada una nueva pila P y un dato $a \in A$, el resultado de aplicar la función devuelve una pila actualizada con el nuevo valor.



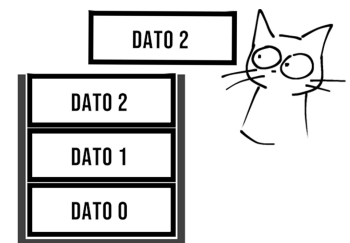
$Pop : Pila[A] \rightarrow Pila[A]$

Esta operación extrae el elemento en el tope de la pila. Como resultado obtenemos la pila orinal sin el último elemento que fue agregado.



$Top : Pila[A] \rightarrow A$

Tras aplicar la operación *Top* a una pila de $P[A]$ obtenemos el dato a que se encuentra en el tope de P sin retirarlo.



$esVacia : Pila[A] \rightarrow Booleano$

Nos arroja *Verdadero* si la pila está vacía y *Falso* en otro caso.



Si a estas operaciones añadimos un constructor, que genere una pila vacía podríamos obtener todo cuanto necesitamos para construir nuestro TDA Pila, que haremos en la siguiente y última sección de esta nota.

El TDA Pila[A]

```
TDA Pila[A]

REQUIERE
    Booleano, A

CONSTANTES
     $Z_0$  : Pila[A]

VARIABLES
    P : Pila[A]
    a : A

OPERACIONES
    crear :  $\emptyset \rightarrow \text{Pila}[A]$ 
    push :  $\text{Pila}[A] \times A \rightarrow \text{Pila}[A]$ 
    pop :  $\text{Pila}[A] \rightarrow \text{Pila}[A]$ 
    top :  $\text{Pila}[A] \rightarrow A$ 
    esVacia :  $\text{Pila}[A] \rightarrow \text{Bool}$ 

AXIOMAS
    [Pa1] crear() =  $Z_0$ 
    [Pa2] pop(crear()) = crear()
    [Pa3] pop(push(P, a)) = P
    [Pa4] top(crear()) = err
    [Pa5] top(push(P, a)) = a
    [Pa6] esVacia( $Z_0$ ) = Verdadero
    [Pa7] esVacia (push (P,e)) = Falso

FIN Pila [A]
```

Listing 6: .]TDA Pila[A].

Observa que Z_0 es el símbolo para la pila vacía, esto es, la pila que no contiene ningún dato. Dicho de otro modo, Z_0 es símbolo que escogimos para denotar el resultado de aplicar la operación `crear()`.

A decir de esta operación, si observamos su definición en el apartado de operaciones, podemos notar que su dominio es el vacío $\emptyset$. Esta es una forma de decir que `crear()` no recibe ningún parámetro. Otra forma de expresar la definición de `crear()` conlleva una total ausencia de símbolo en el lugar designado para el dominio:

$$\text{crear} : \rightarrow \text{Pila}[A]$$

Sin embargo, optamos por el símbolo del conjunto vacío para mayor claridad.

Otra cosa que salta a la vista aparece en el axioma [Pa4] que devuelve `err` si aplicamos la operación `top` a la pila recién creada. ¿Por qué?

Resulta que la pila que no contiene elementos no puede devolver elemento alguno. Sin embargo la operación `top` exige que se devuelva un elemento. Para entender esta situación podríamos remitirnos a un hipotético caso en el que tuviésemos que definir la división de enteros. En tal caso sería necesario añadir un axioma que restrinja la división entre cero, es decir, tendríamos algo como:

$$\text{division}(x, 0) = \text{err}$$

Desde otra perspectiva, lo que queremos expresar es que la función `top` es parcial porque está definida sobre todas las pilas excepto la vacía.

Referencias

- [1] Galaviz Casas, José, *Estructuras de datos y análisis de algoritmos: Una introducción usando Java*. Ciudad de México: Facultad de Ciencias, UNAM.
- [2] J. Prichard, J. M. Carrano, F., y Banerjee, I. *Data Abstraction and Problem Solving with Java: Walls and Mirrors*, 3.^a ed. Boston, MA: Pearson, 2011.